

Distribution-Aware Unionable Table Search

Besat Kassaie
Data Intelligence Lab
Waterloo, Canada
bkassaie@uwaterloo.ca

Renée J. Miller
Data Intelligence Lab
Waterloo, Canada
rjmiller@uwaterloo.ca

ABSTRACT

We introduce Distribution-Aware Unionable Table Search (DUTS), the problem of retrieving a set of unionable tables whose aggregate distribution satisfies user-specified distribution requirements while maximizing unionability.

PVLDB Reference Format:

Besat Kassaie and Renée J. Miller. Distribution-Aware Unionable Table Search. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

Traditional unionable table search retrieves tables that are semantically compatible with a query table, enabling users to augment it with additional tuples. However, semantic compatibility alone is often insufficient. Many real-world applications require retrieved tables to collectively satisfy desired distributional properties over one or more attributes. A data scientist augmenting a training dataset may need to preserve a target class distribution; a healthcare researcher may require patient records whose aggregate demographics match a reference population; a business analyst may demand sales data with a prescribed geographic or product-category breakdown. More broadly, benchmark construction, domain adaptation, and representative data acquisition all impose user-specified distributional constraints that pure semantic similarity cannot capture. These applications motivate a more general retrieval problem, which we term Distribution-Aware Unionable Table Search (DUTS). Given a query table and a data lake of candidate tables, DUTS retrieves a subset that jointly maximizes unionability while ensuring the aggregate distribution of selected tuples satisfies a user-specified distributional objective. Demographic representation is one natural instance of DUTS, but it is a special case of a substantially broader problem class. One obvious instantiation of DUTS is fairness-aware dataset construction, where the distributional objective encodes balance with respect to a protected attribute such as gender or race. A substantial body of prior work addresses this setting by modifying an existing dataset’s distribution post hoc, either through data transformation and reweighting to achieve target fairness criteria at minimum edit cost [BRUNO SCARONE; bias mitigation

citations], or through active tuple acquisition guided by acquisition cost [citation]. Our work departs from this line of research in a fundamental way: rather than correcting distributional skew after data collection, we embed the distributional objective directly into the retrieval process, seeking tables that already exhibit the desired characteristics. This upstream approach reduces or eliminates the need for downstream data modification — an important practical advantage when such modifications are costly, legally constrained, or semantically undesirable. Furthermore, unlike tuple-level acquisition frameworks, we operate at the granularity of tables sourced from a data lake, which reflects the practical reality that raw tuples are rarely available independently of their containing tables. Finally, our approach is entirely schema-agnostic: we make no assumptions about the existence of metadata, schema, or background knowledge about the data lake’s contents. Together, these distinctions, retrieval-time distribution enforcement, table-level granularity, and schema-agnostic operation, set our work apart from existing approaches to fairness-aware and distribution-aware data collection.

To address these challenges, we make the following contributions. We formally define the DUTS problem and prove that it is NP-hard and establishing that efficient exact solutions are unlikely and motivating the need for approximation. We then propose an efficient approximation algorithm, designed to scale to the large candidate sets typical of real-world data lakes. And rather than treating distribution enforcement as a post-retrieval filter, we embed distributional awareness directly into both the index structure and the search algorithm itself, enabling joint optimization of semantic unionability and distributional similarity in a single retrieval pass.

2 NOTATION

A *table* T consists of a schema and a finite multiset of tuples conforming to it. A *data lake* $\mathcal{T} = \{T_1, \dots, T_{|\mathcal{T}|}\}$ is a repository of such tables. For an attribute A , $Dom(A)$ denotes its domain, the set of values A may take.

Given a table T , a distribution attribute $V_D \in T$ is a categorical attribute of T over which a distributional constraint is to be enforced; V_D partitions the tuples of T into groups according to its domain values. Given a table T , a distribution attribute $V_D \in T$, and a set of distribution values $\mathcal{M} \subset Dom(V_D)$, let

$$N(T, V_D, \mathcal{M})$$

denote the number of tuples in T whose value of V_D lies in $\mathcal{M}$, and let

$$F(T, V_D, \mathcal{M}) = \frac{N(T, V_D, \mathcal{M})}{|T|}$$

denote the corresponding proportion. Let $\mathcal{F}^* \in [0, 1]$ denote the desired target proportion for $\mathcal{M}$. Unless otherwise specified, we omit the arguments V_D and $\mathcal{M}$ for brevity, writing $N(T)$ and $F(T)$. We define

$$\Delta(\mathcal{F}^*, F(T)) = \mathcal{F}^* - F(T)$$

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

to measure the signed divergence between the achieved and target proportions; Δ may be negative, permitting $F(T)$ to exceed $\mathcal{F}^*$.¹

Given a query table Q and a data lake table T , let $U(Q, T) \in [0, 1]$ denote the unionability score between Q and T , computed at the level of the whole table with no constraint on which attributes are matched. We separately write $U_{V_D}(Q, T) \in [0, 1]$ for the unionability of Q and T under the constraint that the distributional attribute of Q is aligned with exactly one semantically matching attribute of T ; this constrained score is used where retrieval or filtering depends on V_D -alignment specifically (Section 7). For a candidate set $S \subseteq \mathcal{T}$, we extend U to the set level, conditioned on alignment with the distributional attribute, as the mean per-table score:

$$U_{V_D}(Q, S) = \frac{1}{|S|} \sum_{T_i \in S} U_{V_D}(Q, T_i) \in [0, 1].$$

Every optimization problem in this paper fixes the cardinality of S (to k or to $\alpha \times k$), so maximizing $U_{V_D}(Q, S)$ over sets of a given size is equivalent to maximizing the corresponding unnormalized sum $\sum_{T_i \in S} U_{V_D}(Q, T_i)$; we use the sum form where algorithmically convenient (e.g., Theorem 3.2).

Given a set of tables S , let $\mathbb{T}(S)$ denote the table obtained by taking the multiset union of the tables in S , following the union procedure of [9]. We write

$$F_S := F(\mathbb{T}(S), V_D, \mathcal{M}), \quad N_i := N(T_i, V_D, \mathcal{M}), \quad n_i := |T_i|$$

as shorthand for the achieved proportion of a candidate set's union and for the per-table matching count and size of table T_i ; F_S is used throughout in place of a separately named ratio function.

We determine whether an attribute $A \in T$ is categorical via a threshold θ_{cat} on the number of distinct values it takes. Writing $\text{ndv}(T, A) := |\{t.A : t \in T\}|$ for the number of distinct values of A appearing among the tuples of T , we treat A as categorical whenever $\text{ndv}(T, A) \leq \theta_{\text{cat}}$. The threshold θ_{cat} may be specified by the user or computed automatically from statistics of the data lake $\mathcal{T}$ (e.g., as a percentile of the ndv values observed across all attributes in $\mathcal{T}$). A distribution attribute V_D is required to be categorical in this sense.

Each table $T_i \in D$ is summarized by its empirical distribution over $\text{Dom}(V_D)$, the proportion $H_i[c] \geq 0$ of T_i 's tuples taking value c , with $\sum_{c \in \text{Dom}(V_D)} H_i[c] = 1$, together with its size $n_i = |T_i|$. More generally, indexing is performed at the attribute level, not the table level: a table may have several categorical attributes (Table 1), and each is summarized by its own histogram. We write H_i^A for the histogram of table T_i over categorical attribute $A \in T_i$, so that a table T_i with r categorical attributes is associated with r synopses $H_i^{A_1}, \dots, H_i^{A_r}$, one per attribute; H_i (without a superscript) denotes $H_i^{V_D}$ specifically, the one synopsis relevant to the distributional constraint considered here.

Table 1 summarizes these symbols together with the remaining symbols used throughout the paper.

3 PROBLEM DEFINITION

We consider the single distribution attribute case and develop our framework accordingly. Distribution-Aware Unionable Table Search (DUTS) aims to identify a set R of k unionable tables from a data

¹Fairness is commonly defined as preserving the representation of predefined demographic groups within the selected data, enforced through cardinality constraints on a protected attribute [7]. This notion maps naturally onto our distributional objective F .

Table 1: Notation.

Symbol	Description
$\mathcal{T}$	Data lake: a collection of tables
Q	Query table
V_D	Distributional attribute, $V_D \in Q$
$\mathcal{M}$	Distributional values of interest, $\mathcal{M} \subset \text{Dom}(V_D)$
$N(T, V_D, \mathcal{M}), N_i$	Number of tuples in T (resp. T_i) whose V_D -value lies in $\mathcal{M}$
n_i	Number of tuples in T_i , i.e., $ T_i $
$F(T, V_D, \mathcal{M})$	Achieved proportion, $N(T, V_D, \mathcal{M})/ T $
$\mathcal{F}^*$	Target proportion for $\mathcal{M}$
$\Delta(\mathcal{F}^*, F(T))$	Signed deviation, $\mathcal{F}^* - F(T)$
δ	Maximum allowable deviation, $\Delta \leq \delta$
S, R	A candidate set of tables; the returned result set
k	Required size of the result set, $ R = k$
α	Pool-expansion factor used in Stage 1 (Section 4)
$\mathbb{T}(S)$	Table obtained by the multiset union of the tables in S
F_S	Shorthand for $F(\mathbb{T}(S), V_D, \mathcal{M})$
$U_{V_D}(Q, T)$	Unionability of Q and T under the constraint that the distributional attribute of Q is aligned with exactly one semantically matching attribute of T
$U(Q, T)$	Unionability score between Q and table T
$U_{V_D}(Q, S)$	Mean unionability of a set S with Q , $\frac{1}{ S } \sum_{T_i \in S} U_{V_D}(Q, T_i)$
$\text{ndv}(T, A)$	Number of distinct values attribute A takes among the tuples of T
θ_{cat}	Threshold on ndv used to determine whether an attribute is treated as categorical
H_i^A	Histogram (empirical distribution) of table T_i over categorical attribute $A \in T_i$; $H_i := H_i^{V_D}$
$\text{sim}(A, V_D)$	Embedding similarity between attribute A and the query's distributional attribute V_D , as computed by the HNSW index (Section 7.1)
R_{vi}	Subset of R whose tables individually violate the distributional constraint, i.e., $R_{vi} = \{T_i \in R : N_i/n_i < \tau^*\}$
C	Pool of replacement candidate tables considered for repairing R
σ	A repair: a partial injective matching from R_{vi} into C
$R^{(\sigma)}$	Result of applying repair σ to R
$L(\sigma)$	Unionability lost by repair σ , $U_{V_D}(Q, R) - U_{V_D}(Q, R^{(\sigma)})$

lake that maximizes unionability with Q while ensuring that the representation of tuples associated with the distribution attribute V_D satisfies the user-specified target proportion. Unlike traditional unionability search, our formulation constrains the candidate tables to those that contain the distribution attribute and ensures that distribution considerations are incorporated directly into the search process. Consequently, if there is no unionable table in the data lake whose attributes can be matched with V_D , the search returns the empty set.

Definition 3.1 (Distribution-Aware Unionable Table Search). Given a data lake $\mathcal{T}$, a query table Q , a distribution attribute $V_D \in Q$, a set of distribution values $\mathcal{M}$, a unionability scoring function U_{V_D} ,

and a result size k , the distribution-aware unionable table search problem is defined as:

$$R = \arg \max_{S \subseteq \mathcal{T}, |S|=k} U_{V_D}(Q, S) \quad (1)$$

s.t. $\Delta(\mathcal{F}^*, F(\mathbb{T}(S \cup Q), V_D, \mathcal{M})) \leq \delta,$

where $\delta \in [0, 1]$ denotes the maximum allowable deviation from the desired distribution (Table 1).

The objective is determined by unionability, but the solution space is restricted by a distributional constraint, in a formulation similar to that of [3]. Recall that Δ is signed, so it may be negative, permitting the achieved proportion F to exceed the target $\mathcal{F}^*$.

Note that the distribution constraint in Definition 3.1 is not imposed on each returned table individually. Rather, it is enforced collectively: when the records from the selected tables are aggregated by $\mathbb{T}(S \cup Q)$, the resulting combined population must satisfy the distributional requirement, i.e., its representation must not deviate beyond the allowed threshold.

F_S is inherently *aggregation-sensitive*: the aggregated proportion is not equivalent to any arithmetic combination of the tables' local proportions. Consequently, larger tables can disproportionately obscure the distributional characteristics of smaller tables. This distinction is particularly important in data lakes, where unionable tables may vary substantially in size and distribution.

A straightforward solution for finding R requires enumerating all subsets of size k , computing both their deviation and unionability scores, and selecting the subset that optimizes the objective. This exhaustive search is computationally expensive in practice. Next, we show that the problem is, in fact, NP-hard.

THEOREM 3.2. *Problem 3.1 is NP hard.*

PROOF SKETCH. We reduce from the Exact k -Item Knapsack Problem (E- k KP), which is known to be NP-complete [2]. An instance of E- k KP consists of items with profits p_j , weights w_j , a capacity c , a required cardinality k , and a target profit V . The objective is to select exactly k items with total weight at most c while maximizing the total profit. For each item j , we construct one table T_j containing exactly c tuples, of which $c - w_j$ contain values from $\mathcal{M}$ and the remaining w_j do not. Consequently, the table size is $n_j = c$, the number of tuples matching the distributional values is $N_j = c - w_j$, and the unionability score is set to $u_j = p_j$. Thus, all tables have the same size, while heavier items correspond to fewer tuples matching $\mathcal{M}$.

Consider any subset S of exactly k tables. Since every table has size c , the achieved distribution is

$$F_S = \frac{\sum_{T_j \in S} N_j}{\sum_{T_j \in S} n_j} = \frac{\sum_{T_j \in S} (c - w_j)}{kc} = 1 - \frac{\sum_{T_j \in S} w_j}{kc}.$$

Since

$$F_S = 1 - \frac{\sum_{T_j \in S} w_j}{kc},$$

Choosing

$$\mathcal{F}^* - \delta = \frac{k-1}{k},$$

the distribution constraint becomes

$$\begin{aligned} F_S \geq \mathcal{F}^* - \delta &\iff 1 - \frac{\sum_{T_j \in S} w_j}{kc} \geq \frac{k-1}{k} \\ &\iff \sum_{T_j \in S} w_j \leq c, \end{aligned}$$

which is exactly the knapsack capacity constraint. Moreover, the unionability objective satisfies

$$U_{V_D}(Q, S) = \frac{1}{k} \sum_{T_j \in S} u_j = \frac{1}{k} \sum_{T_j \in S} p_j,$$

which differs from the knapsack profit only by the constant factor $1/k$. Therefore, maximizing unionability is equivalent to maximizing profit.

Consequently, there is a one-to-one correspondence between feasible solutions of E- k KP and feasible solutions of Problem 3.1, and the objective values differ only by the constant factor $1/k$. Therefore, an optimal solution to Problem 3.1 results in an optimal solution to the corresponding E- k KP instance. Since the reduction is computable in polynomial time and E- k KP is NP-hard, the Problem 3.1 is NP-hard. $\square$

4 SOLUTION OVERVIEW

Theorem 3.2 establishes that Problem (1) is NP-hard. Examining the reduction provides insight into the source of this hardness and directly motivates our algorithmic design. In the reduction, the distributional constraint $\Delta(\mathcal{F}^*, F(\cdot)) \leq \delta$ is transformed into a knapsack capacity constraint, while the unionability scores $U_{V_D}(Q, \cdot)$ correspond to item profits. Consequently, the hardness does not stem from evaluating the distributional criterion itself, but from maximizing unionability under a fixed distributional constraint, yielding the combinatorial structure of an exact- k knapsack problem.

4.1 A Two-Stage Framework.

This observation naturally raises the question of whether the distributional criterion is itself difficult to optimize in the absence of the unionability objective. We show in Section 5 that it is not. Optimizing $F(\mathbb{T}(S \cup Q), V_D, \mathcal{M})$ can be formulated as a linear-fractional optimization problem over the table-selection variables and solved exactly using Dinkelbach's parametric method [6]. In our setting, over a set of n tables each Dinkelbach iteration requires $O(n \times \log n)$ time, yielding an overall running time of $O(I \times n \times \log n)$, where I denotes the number of Dinkelbach iterations. In practice, Dinkelbach's algorithm typically converges in only a few iterations [12].

We exploit this property to develop a two-stage optimization framework. Given the desired result size k , the first stage ignores unionability and computes the top αk candidate tables that maximize the distributional objective, where $\alpha \geq 1$ is a user-defined expansion factor controlling the size of the candidate pool. The second stage then solves Problem (1) over these $\alpha \times k$ candidates rather than over the entire repository $\mathcal{T}$, selecting the final k tables. By confining the NP-hard optimization to a reduced search space, our framework significantly lowers the computational cost while allowing the user to control the trade-off between efficiency and solution quality through the expansion factor α .

Stage 1: Distribution-aware candidate selection. The first stage constructs an expanded candidate pool $P \subseteq \mathcal{T}$ of size $\alpha \times k$, where $\alpha > 1$ is a user-defined expansion factor, by optimizing the distributional criterion:

$$P = \arg \max_{S \subseteq \mathcal{T}, |S|=\alpha \times k} F(\mathbb{T}(S \cup Q), V_D, \mathcal{M}).$$

The resulting candidate pool serves as the search space for the second stage. Since it is selected to maximize the distributional objective, it concentrates the subsequent combinatorial optimization on tables that are most promising with respect to the desired distribution, thereby substantially reducing the search space while preserving a diverse set of high-quality candidates.

Stage 2: Unionability-aware selection. The second stage solves Problem (1) over the candidate pool P rather than the full repository $\mathcal{T}$:

$$R = \arg \max_{S \subseteq P, |S|=k} U_{V_D}(Q, S) \quad \text{s.t.} \quad \Delta(\mathcal{F}^*, F(\mathbb{T}(S \cup Q), V_D, \mathcal{M})) \leq \delta. \quad (2)$$

The intuition is simple: tables that best satisfy the distributional objective are more likely to participate in feasible high-quality solutions. Restricting the search to P therefore focuses the expensive unionability optimization on the most promising candidates. This restriction does not change the problem’s NP-hardness; it only reduces the search space. Since $|P| = \alpha \times k$, Stage 2 operates on a user-controlled candidate pool independent of $|\mathcal{T}|$. As a result, unionability scores are computed only for the tables in P , making the optimization in Section 6 practical.

Why optimize the distributional criterion first? The ordering of the two stages is deliberate. A natural alternative is to first retrieve the most unionable tables and subsequently adjust the selection to satisfy the desired distributional constraint, for example by iteratively replacing tables that violate the constraint. We investigate such a greedy swap strategy experimentally in Section 8. However, high unionability alone provides no guarantee that the resulting candidate pool contains a high-quality, or even feasible, solution with respect to the distributional objective. When this occurs, additional candidates must be retrieved and evaluated, increasing the number of expensive query-time unionability computations.

Our framework instead first constructs a candidate pool optimized solely for the desired distribution. The second stage then optimizes unionability only within this reduced pool. Although this heuristic does not eliminate the NP-hardness of the original optimization problem, it substantially reduces the search space and confines expensive unionability computations to a small, user-controlled candidate set.

Sections 5 and 6 describe the two stages in detail.

4.2 Two orthogonal notions of relevance

Searching every attribute in a large data lake for a distributional match is unnecessary. A candidate attribute is relevant only if it satisfies two independent conditions: it is semantically aligned with the query’s distributional attribute, and it shares at least one value from the query’s distributional vocabulary. Efficient index structures exist for both signals. We therefore retrieve candidate attributes by jointly probing these indexes, producing a small candidate set D on which the subsequent optimization operates. The

retrieval stage is driven solely by the query’s distributional attribute V_D . Rather than indexing entire tables, we index only categorical attributes satisfying $\text{ndv}(T_j, A) \leq \theta_{\text{cat}}$ (Table 1). Tables that contain no categorical attribute satisfying the threshold simply cannot contribute to the distribution-aware retrieval stage and are therefore excluded from the index. For every indexed categorical attribute, we precompute and store an embedding, its set of distinct values, and a histogram synopsis. The embedding and value set are organized in dedicated index structures for semantic and value-overlap retrieval, respectively. Histogram synopses are stored separately and consulted only for attributes returned by the retrieval stage.

A categorical attribute is considered a plausible match for the query’s distributional attribute according to two complementary notions of relevance.

- **Semantic relevance.** The candidate attribute should describe the same real-world concept as V_D , even if the two attributes use different names or value encodings (e.g., `nationality` and `country_of_origin`).
- **Value overlap.** The candidate attribute must share at least one value from the query’s distributional vocabulary $\mathcal{M}$. Without such overlap, the attribute cannot contribute to the distributional objective and may be safely discarded.

These two signals capture orthogonal aspects of relevance. Two attributes may be semantically equivalent while sharing no values, or they may share values despite representing different concepts. Consequently, our retrieval procedure evaluates both signals independently and retains only attributes that satisfy both criteria. The resulting tables constitute the candidate set D , which serves as the input to the optimization stage. Both $\mathcal{M}$ and δ are specified at query time and are therefore unavailable during index construction. Details of the index construction are presented in Section 7.

5 DISTRIBUTION-AWARE CANDIDATE RETRIEVAL

The first stage operates on the candidate set $D \subseteq \mathcal{T}$ returned by the retrieval procedure described in Sections 4.2 and 7. At this stage, selection is driven only by the query’s distributional attribute V_D ; the remaining attributes are incorporated later during the unionability optimization of Stage 2. A table is included in D only if it contains an attribute that (i) is semantically aligned with V_D and (ii) shares at least one value from the query’s distributional vocabulary $\mathcal{M}$.

Given the distributional vocabulary $\mathcal{M} \subseteq \text{Dom}(V_D)$ and the desired candidate pool size $\alpha \times k$, Stage 1 seeks the exact- $(\alpha \times k)$ subset $S \subseteq D$ maximizing the achieved distributional proportion:

$$F_S = \frac{\sum_{i \in S} N_i}{\sum_{i \in S} n_i}, \quad N_i = \sum_{c \in \mathcal{M}} n_i H_i[c],$$

where N_i denotes the number of tuples in table T_i whose V_D value belongs to $\mathcal{M}$ (Table 1). The optimal subset S^* forms the distribution-aware candidate pool passed to Stage 2.

The following observation forms the basis of our optimization. The distributional score of a candidate set, F_S , can be expressed exactly as a weighted average of the individual table scores $F_i := F(T_i)$, with weights proportional to the table cardinalities n_i . Starting from

the definition,

$$F_S = \frac{\sum_{i \in S} N_i}{\sum_{i \in S} n_i}.$$

Writing $N_i = F_i n_i$ (by definition of F_i) and substituting,

$$F_S = \frac{\sum_{i \in S} F_i n_i}{\sum_{i \in S} n_i} = \sum_{i \in S} w_i F_i, \quad w_i := \frac{n_i}{\sum_{j \in S} n_j}.$$

Since $w_i \geq 0$ for all $i \in S$ and $\sum_{i \in S} w_i = 1$, the distributional score F_S is exactly the weighted average of the individual table scores $\{F_i\}_{i \in S}$, where each table is weighted by its cardinality.

Consequently, the maximizing subset is *not* simply the $\alpha \times k$ tables with the largest individual F_i : a table with a moderate F_i but favorable n_i can shift the weighted combination more than a table with a higher F_i but small n_i . This is precisely the interaction that ordinary top- $(\alpha \times k)$ ranking by F_i fails to account for, and is what necessitates the parametric search described next.

5.0.1 Formulation as 0-1 fractional programming. Let D denote the candidate set returned by index probing, where $m := |D|$. Each table $T_i \in D$, $i = 1, \dots, m$, is associated with its matched distributional attribute and represented by the statistics (N_i, n_i) , where N_i is the number of tuples whose distributional attribute value belongs to $\mathcal{M}$, and n_i is the cardinality of T_i . We introduce a binary indicator vector $\mathbf{x} = (x_1, \dots, x_m) \in \{0, 1\}^m$, where $x_i = 1$ denotes that table T_i is included in the candidate subset S . The union proportion can then be written directly as a function of $\mathbf{x}$:

$$F(\mathbf{x}) = \frac{\mathbf{N}^\top \mathbf{x}}{\mathbf{n}^\top \mathbf{x}} = \frac{\sum_{i=1}^m N_i x_i}{\sum_{i=1}^m n_i x_i},$$

where $\mathbf{N} = (N_1, \dots, N_m)$ and $\mathbf{n} = (n_1, \dots, n_m)$. Both the numerator and denominator are linear functions of $\mathbf{x}$. Consequently, $F(\mathbf{x})$ is a linear-fractional objective over binary decision variables, yielding a 0-1 fractional program. Our optimization problem is then

$$\max_{\mathbf{x} \in \{0,1\}^m} F(\mathbf{x}) \quad \text{s.t.} \quad \mathbf{1}^\top \mathbf{x} = \alpha \times k,$$

i.e., maximize this linear-fractional objective subject to the cardinality constraint that exactly $\alpha \times k$ entries of $\mathbf{x}$ equal 1.

Why this class of problem is generally hard to optimize directly, since Unlike a linear objective, $F(\mathbf{x})$ does not decompose into a sum of independent per-table contributions, so it cannot be optimized by a direct greedy or sorting argument on $\mathbf{x}$ alone: the contribution of including table i depends on which *other* tables are already included in S , since both numerator and denominator change jointly. This is exactly the interaction highlighted by the weighted-average argument above—the identity of the optimal size- $(\alpha \times k)$ subset depends on the n_i -weighted interaction between candidates, not on their individual F_i values in isolation.

Linear fractional programs were first solved using a one-shot linear programming transformation proposed by Charnes and Cooper [4]. Later, Dinkelbach [6] introduced a parametric algorithm that has been shown to be highly effective in practice for solving linear fractional programs [11, 12]. We adopt Dinkelbach’s method in this work.

Dinkelbach’s parametric method. Dinkelbach’s algorithm resolves this by introducing an auxiliary parameter $\lambda \in \mathbb{R}$ and considering

the *parametric* objective

$$\Phi(\lambda) = \max_{\mathbf{x} \in \{0,1\}^m, \mathbf{1}^\top \mathbf{x} = \alpha \times k} (\mathbf{N}^\top \mathbf{x} - \lambda \mathbf{n}^\top \mathbf{x}) = \max_{|S| = \alpha \times k} \sum_{i \in S} (N_i - \lambda n_i).$$

This subtraction of the denominator, scaled by λ , from the numerator converts the original *ratio* objective $F(\mathbf{x})$ into an ordinary *linear* objective in $\mathbf{x}$, for a fixed value of λ . Because the objective $\sum_{i \in S} (N_i - \lambda n_i)$ is now additively separable across tables, its maximizer over subsets of size exactly $\alpha \times k$ is trivial to compute: it is simply the $\alpha \times k$ tables with the largest values of $y_i(\lambda) := N_i - \lambda n_i$, obtainable via a single top- $(\alpha \times k)$ selection.

The governing theorem. The classical result underlying Dinkelbach’s method [?] is that $\Phi(\lambda)$, viewed as a function of λ alone, is continuous, strictly decreasing, and convex, and that

$$\Phi(\lambda) = 0 \iff \lambda = \max_{|S| = \alpha \times k} F_S.$$

That is, the optimal achievable proportion λ^* is exactly the unique root of Φ . This reduces our original fractional maximization problem to a univariate root-finding problem over $\Phi(\lambda)$, each evaluation of which requires solving one *linear* (not fractional) subproblem—the top- $(\alpha \times k)$ selection described above.

The iterative procedure. Root-finding on Φ is carried out by the following fixed-point iteration:

- (1) Initialize $\lambda^{(0)}$ (e.g., the proportion achieved by the full candidate pool, $\lambda^{(0)} = \mathbf{N}^\top \mathbf{1} / \mathbf{n}^\top \mathbf{1}$).
- (2) At iteration t , compute $y_i(\lambda^{(t)}) = N_i - \lambda^{(t)} n_i$ for all i , and let $S^{(t)}$ be the $\alpha \times k$ tables with the largest $y_i(\lambda^{(t)})$.
- (3) Update $\lambda^{(t+1)} = F_{S^{(t)}} = (\sum_{i \in S^{(t)}} N_i) / (\sum_{i \in S^{(t)}} n_i)$ —i.e., the proportion actually achieved by the subset just selected.
- (4) If $\lambda^{(t+1)} = \lambda^{(t)}$ (or the change falls below a numerical tolerance), terminate: $S^{(t)}$ is the exact global optimum and $\lambda^{(t)} = \max_{|S| = \alpha \times k} F_S$; this optimum is exactly the pool P of Section 4. Otherwise, set $\lambda \leftarrow \lambda^{(t+1)}$ and repeat.

Because $\Phi(\lambda)$ is strictly decreasing and convex, this iteration is guaranteed to converge monotonically to the unique root λ^* , and does so in practice;

6 STAGE 2: UNIONABILITY-AWARE SELECTION

Stage 1 considers only the query’s distributional attribute in order to identify a distribution-aware candidate pool. Stage 2 evaluates the full unionability of these candidates with the query table by considering all remaining attributes. Specifically, for each candidate table, we compute its unionability score with the query based on the semantic similarity of their aligned attributes. These scores form the objective of the optimization problem introduced in Section 4. Given the Stage-1 pool P (Section 5) and its achieved proportion F_P , we address the second stage of Problem (1) (Equation (2)): select $R \subseteq P$, $|R| = k < wk$, obtained by removing $wk - k$ tables from P , that maximizes total unionability $\sum_{T_i \in R} U(Q, T_i)$ —equivalent, since $|R| = k$ is fixed, to maximizing the mean $U_{V_D}(Q, R)$ of Equation (1) directly (Table 1)—while respecting a final constraint on F_R . Only the final state at size k is constrained; intermediate subset sizes encountered while removing tables are unconstrained.

6.1 Optimization Formulation

Let $\tau := \mathcal{F}^* - \delta$ denote the proportion floor implied by the feasibility constraint $\Delta(\mathcal{F}^*, F(\cdot)) \leq \delta$ (Table 1). The reduction problem is

$$R = \arg \max_{S \subseteq P, |S|=k} \sum_{T_i \in S} U_{V_D}(Q, T_i) \quad \text{s.t.} \quad \frac{\sum_{T_i \in S} N_i}{\sum_{T_i \in S} n_i} \geq \tau,$$

matching Equation (2) with the constraint linearized at the fixed floor τ . Unlike the optimization of Section 5, the objective here is a plain linear sum (unionability), and the proportion constraint is fixed rather than itself being optimized. Substituting $y_i(\tau) = N_i - \tau n_i$, the constraint becomes the single linear inequality $\sum_{T_i \in S} y_i(\tau) \geq 0$, precomputed once per candidate since τ is fixed. Candidates are drawn only from P ; the reduction step removes tables from the Stage-1 pool rather than reconsidering tables outside it.

6.1.1 Solution via Integer Linear Programming. Because τ is fixed and the objective is linear in the decision variables, the reduction problem can be formulated as a 0–1 integer linear program (ILP). Introducing binary decision variables $\mathbf{x} \in \{0, 1\}^{|P|}$ over the candidate tables,

$$\max_{\mathbf{x} \in \{0, 1\}^{|P|}} \sum_{T_i \in P} U_{V_D}(Q, T_i) x_i$$

subject to

$$\sum_{T_i \in P} y_i(\tau) x_i \geq 0, \quad \sum_{T_i \in P} x_i = k,$$

where $y_i(\tau) = N_i - \tau n_i$. The formulation consists of a linear objective and two linear constraints over binary decision variables. Since the optimization is performed over the Stage-1 candidate pool, whose size is bounded by ak (Section 4), the number of binary variables is independent of the size of the data lake, making exact optimization practical using a standard ILP solver.

As a preprocessing step, we first solve the LP relaxation obtained by replacing the binary constraints $x_i \in \{0, 1\}$ with $x_i \in [0, 1]$. If the relaxation is infeasible, then the ILP is also infeasible, implying that no size- k subset of P satisfies the distributional constraint. Otherwise, the ILP is solved to obtain the exact optimal subset R .

Section ?? reports the execution-time savings achieved by this preprocessing step by comparing direct ILP solving against solving the LP relaxation prior to the ILP. Implementation details, including the ILP solver used, are provided in the experimental setup.

6.2 Constrained Attribute Alignment

Stage 2 computes the unionability between the query table Q and each candidate table T by solving an attribute alignment problem. Following the graph construction of Starmie [8], we construct a weighted bipartite graph whose vertices represent the attributes of the query and candidate tables. Each edge $e = (x, y) \in E$ is assigned the pairwise attribute similarity score $s(x, y)$ between attributes x and y . As in Starmie, we retain only edges whose pairwise attribute similarity score is at least the threshold σ , using the same value of the threshold as in the experimental evaluation of Starmie.

Unlike Starmie, however, our framework assumes that the alignment of the query’s distributional attribute V_D has already been determined during the candidate generation stage. Consequently, the matched pair (V_D, V_{DT}) is fixed and excluded from the optimization, where V_{DT} denotes the attribute of T matched to V_D during

candidate generation. Let $G = (X \cup Y, E)$ denote the resulting bipartite graph over the remaining attributes of the query table (X) and the candidate table (Y). We compute a maximum-weight bipartite matching M^* over G . The unionability score between Q and T is defined as the sum of the pairwise attribute similarity scores of all aligned attribute pairs, including the fixed distributional-attribute alignment:

$$U_{V_D}(Q, T) = s(V_D, V_{DT}) + \sum_{(x, y) \in M^*} s(x, y),$$

7 INDEXING AND CANDIDATE RETRIEVAL

7.1 Semantic filtering via HNSW

Each table’s distributional attribute V_D is embedded into a fixed-dimensional vector using a pretrained encoder over its values, and all such embeddings are indexed with HNSW [10], a graph-based approximate nearest-neighbor (ANN) structure offering logarithmic expected query time and high recall in practice. At query time, we issue a top- N ANN query against the query table’s V_D -embedding and retain the neighbors whose Euclidean distance falls below a threshold θ_{dis} ; because HNSW is natively a top- N retrieval structure, over-retrieving and threshold-filtering the returned list is the standard way to realize a similarity cutoff without sacrificing the index’s query-time guarantees. HNSW trades exhaustive recall for query speed, so it can occasionally err in two ways: it may miss a genuinely relevant table (a false negative), or it may return one that turns out not to be a good match (a false positive). These two errors are not equally costly. A missed table never enters the candidate pool D , so the optimizer never has the chance to consider it – a real, if bounded, loss of solution quality. A wrongly admitted table, by contrast, costs nothing in the end: it still has to clear the JOSIE overlap check, and its exact counts N_i are verified before the optimizer can select it, so it is simply discarded once its true value becomes clear. Because only false negatives can degrade the final answer, while false positives are caught and corrected downstream, we accept HNSW’s approximate recall as a favorable trade for its query-time speed.

7.2 Value-overlap filtering via JOSIE

In parallel, we index the categorical values of every table’s distributional attribute with JOSIE [13], which supports exact top- k overlap search over set-valued columns using inverted lists and a cost-based search strategy. Querying JOSIE with the query table’s distributional values of interest $\mathcal{M}$ returns, for every candidate with nonzero overlap, the exact overlap size. We retain candidates with overlap size at least 1 – the necessary condition motivated above.

7.3 Combining the Two Indexes

The two filters above are applied only to the query’s distributional attribute V_D : HNSW is probed on V_D ’s embedding, and JOSIE is queried with V_D ’s value set of interest $\mathcal{M}$. No non-distributional attribute of a candidate table participates in retrieval at this stage; their contribution to unionability is assessed later, once candidates are fixed, by comparing each aligned attribute pair directly.

Because the two filters operate at the attribute level, each returns a set of *(table, attribute)* pairs rather than a set of tables: $D_{\text{sem}} \subseteq \{(T_j, A)\}$ is the HNSW-filtered neighbor set of Section 7.1, and $D_{\text{ovl}} \subseteq \{(T_j, A)\}$ is the set of attribute candidates JOSIE returns with nonzero overlap (Section 7.2). We first combine them by intersecting at the pair level,

$$D_{\text{pair}} = D_{\text{sem}} \cap D_{\text{ovl}},$$

so that a pair (T_j, A) survives only if the *same* attribute A is judged a plausible semantic match for V_D and provably shares at least one value in M ; a table entering D_{sem} via one attribute and D_{ovl} via a different attribute of the same table does not, by this construction, produce a surviving pair.

Pair-level intersection does not yet guarantee a one-to-one mapping between tables and attributes: a table T_j may possess several categorical attributes (Table 1), more than one of which can independently satisfy both filters—for instance, a table with both a nationality and a citizenship attribute, either of which may plausibly align with V_D . We resolve any such redundancy by keeping, for each table, only its most semantically similar surviving attribute, which is computed over attribute’s embedding during HNSW retrieval (Section 7.1). We use semantic similarity rather than overlap size to break these ties because overlap is used throughout only as a hard, boolean necessary condition (Section 4.1), any nonzero overlap suffices to clear that bar, whereas HNSW similarity is a continuous, already-available ranking signal naturally suited to selecting among several attributes that each clear both bars.

Computing D_{pair} and resolving ties by semantic similarity is inexpensive: both D_{sem} and D_{ovl} are already small, index-retrieved sets, and the intersection and per-table argmax are computed once, without any further index probe.

8 AN ALTERNATIVE DESIGN: SWAP-BASED DISTRIBUTIONAL REPAIR

Incorporating distributional awareness into unionable table search introduces three key trade-offs:

- (1) The unionability score of the search result can be adversely affected for two reasons. First, we restrict candidate tables to those with exactly one attribute aligned with V_D , which may exclude otherwise highly unionable tables. Second, even a highly unionable table may be removed if it violates the distributional constraint.
- (2) Enforcing the distributional constraint introduces additional computation, which can increase execution time and degrade runtime performance.
- (3) The search may return an empty result when no feasible solution exists, even if a highly unionable set of tables is available.

We seek to control and reduce the impact of these effects on both result quality and runtime, while allowing an empty result when no feasible solution is found.

As discussed earlier, our framework operates on the relatively large candidate set returned by an existing unionable table search system, Starmie [8]. Restricting the optimization to this candidate set substantially reduces the search space from the entire data lake to a manageable number of tables, enabling efficient optimization. The cost of this design choice is that optimality is conditioned on

the quality of the initial retrieval; if the globally optimal set of tables is absent from the candidate set, it cannot be recovered by our optimization.

Given a set of tables of size wk (Table 1), we partition the search space into two components: (i) a set R of size k , consisting of the tables with the highest unionability scores whose union violates the distributional constraint, and (ii) a candidate pool C containing possible replacement tables whose unionability scores are less than or equal to those of the tables in R . Our objective is to replace tables in R with tables from C such that the resulting union satisfies the constraint while minimizing the loss in unionability.

We next show that to achieve the distributional requirement it suffices to focus exclusively on tables that individually violate the constraint. More precisely, if no individual table in R violates the constraint, then the union $\mathbb{T}(R)$ cannot violate it either. This locality property drives our repair strategy: it allows replacements to be restricted to the violating subset $R_{vi} \subseteq R$ rather than all of R , reducing the search space.

CLAIM 1. (Locality of Violations) *Let R be a set of tables and let V_D be the distribution attribute present in every table in R . Given a distributional target $\mathcal{F}^*$ and a maximum allowable deviation δ , if the union $\mathbb{T}(R)$ violates the distributional constraint, then there exists at least one table $T_j \in R$ that individually violates the constraint.*

PROOF. Let $\tau^* = \mathcal{F}^* - \delta$. We wish to show that

$$F_R = \frac{\sum_{T_i \in R} N_i}{\sum_{T_i \in R} n_i} < \tau^* \implies \exists T_j \in R : \frac{N_j}{n_j} < \tau^*.$$

Assume by contradiction that every table satisfies the distributional constraint individually:

$$\forall T_i \in R : \frac{N_i}{n_i} \geq \tau^* \implies N_i \geq \tau^* n_i.$$

Summing over all tables in R yields

$$\sum_{T_i \in R} N_i \geq \tau^* \sum_{T_i \in R} n_i,$$

which gives $F_R \geq \tau^*$, contradicting the assumption $F_R < \tau^*$. $\square$

Therefore, by Claim 1, it suffices to consider only the subset $R_{vi} \subseteq R$, consisting of the tables that violate the distributional constraint individually. This shift might reduce the number of tables considered for a swap; however, we still face the complexity problem addressed below.

Repairing a Violating Union. Given a result set R whose union violates the distributional constraint, we seek to restore feasibility by replacing some violating tables with candidate tables. Since candidates are retrieved after the initial unionability ranking, they are assumed to be no more unionable than the selected tables. Consequently, every repair trades unionability for improved balance.

Let

$$R_{vi} = \left\{ T_i \in R \mid \frac{N_i}{n_i} < \tau^* \right\}, \quad \tau^* = \mathcal{F}^* - \delta,$$

denote the violating tables. Let C denote the candidate pool, where

$$U_{V_D}(Q, c) \leq \min_{T_i \in R} U_{V_D}(Q, T_i), \quad \forall c \in C.$$

A repair selects a subset $A \subseteq R_{vi}$ of violating tables to remove and an equally sized subset $B \subseteq C$ of candidate tables to insert. The repaired solution is

$$R' = (R \setminus A) \cup B,$$

where $|A| = |B|$. Consequently, $|R'| = |R| = k$.

The repair cost is the loss in average unionability,

$$L(A, B) = U_{V_D}(Q, R) - U_{V_D}(Q, R').$$

Definition 8.1 (CHEAPEST REPAIR). Given a violating solution R , candidate pool C , and target distribution τ^* , find subsets $A \subseteq R_{vi}$ and $B \subseteq C$ with $|A| = |B|$ minimizing

$$L(A, B)$$

subject to

$$F(\mathbb{T}(R'), V_D, \mathcal{M}) \geq \tau^*,$$

where

$$R' = (R \setminus A) \cup B.$$

A natural approach is to greedily select the candidates with the highest individual contribution to the distributional balance. However, the distributional constraint is inherently *non-additive*: whether the repaired union satisfies the target threshold τ^* depends on the pooled distribution of the selected candidates rather than on their individual scores. Consequently, no local ranking of candidates is sufficient to identify an optimal repair, and the optimization must reason over candidate subsets.

Example 8.2 (Greedy subset selection fails). Let

$$\tau^* = 0.75, \quad R_{vi} = \{t_1, t_2\}, \quad C = \{c_1, c_2, c_3\},$$

where

	$U(Q, \cdot)$	N	$n - N$	n	F
t_1	10	1	9	10	0.10
t_2	10	3	7	10	0.30
c_1	8.0	2	0	2	1.00
c_2	9.0	4	2	6	0.67
c_3	9.5	14	6	20	0.70

Assume both violating tables must be replaced, i.e., $|A| = |B| = 2$. Selecting candidates greedily by unionability chooses

$$B = \{c_2, c_3\},$$

whose pooled distribution is

$$\frac{14 + 4}{20 + 6} = \frac{18}{26} \approx 0.69 < 0.75.$$

Selecting greedily by individual distribution instead chooses

$$B = \{c_1, c_3\},$$

giving

$$\frac{2 + 14}{20 + 20} = \frac{16}{40} \approx 0.73 < 0.75.$$

The only feasible repair is obtained by selecting

$$B = \{c_1, c_2\},$$

whose pooled distribution satisfies

$$\frac{2 + 4}{2 + 6} = \frac{6}{8} = 0.75.$$

This example illustrates that feasibility depends on the joint effect of the selected candidate subset rather than on individual unionability or distribution scores.

Example 8.2 illustrates that CHEAPEST REPAIR is inherently combinatorial. Neither unionability nor the individual distribution score of a candidate is sufficient to determine whether a subset of candidates satisfies the distributional constraint. Feasibility depends on the collective effect of the selected candidates, requiring the optimization to search over candidate subsets rather than make independent greedy choices.

THEOREM 8.3. *Computing a minimum-cost repair is NP-hard.*

PROOF. We reduce from SUBSET SUM. Given positive integers $a_1, \dots, a_n$ and target $W \leq \sum_{i=1}^n a_i$, the goal is to determine whether there exists a subset whose sum is exactly W .

Let $M = \sum_{i=1}^n a_i$. For each item a_i , construct one violating table $t_i \in R_{vi}$ and one candidate table $c_i \in C$ satisfying

$$N(t_i) = 0, \quad n(t_i) = M, \quad U(Q, t_i) = 0,$$

and

$$N(c_i) = a_i, \quad n(c_i) = M, \quad U(Q, c_i) = -a_i.$$

Since $0 \leq a_i \leq M$, every constructed table satisfies $0 \leq N(\cdot) \leq n(\cdot)$ and is therefore a valid instance.

Let $R = R_{vi} = \{t_1, \dots, t_n\}$ and $k = n$, and choose $\mathcal{F}^*$ and δ such that $\tau^* = \mathcal{F}^* - \delta = W/(nM)$; since $0 \leq W \leq M \leq nM$, we have $\tau^* \in [0, 1]$. The construction stores the value M symbolically and is computable in polynomial time.

A repair is specified by a subset $B \subseteq C$ together with a same-size subset $A \subseteq R_{vi}$, $|A| = |B|$, of tables to remove, yielding $R' = (R \setminus A) \cup B$. Because $U(Q, t_i) = 0$ for every $t_i \in R_{vi}$, the removal cost $\sum_{t_i \in A} U(Q, t_i) = 0$ regardless of which $|B|$ violating tables are chosen for A ; since $|R_{vi}| = n \geq |B|$ always holds, some valid choice of A exists for every $B \subseteq C$, and all such choices yield the same R' up to which violating tables remain in it, hence the same $n(R')$, $F(\mathbb{T}(R'))$, and cost. We may therefore treat $B \subseteq C$ alone as specifying the repair, with A any $|B|$ -element subset of R_{vi} .

Because every table has exactly M tuples, replacing tables preserves the total tuple count: $n(R') = n(R) = nM$, independent of B . Moreover, every violating table remaining in R' contributes zero tuples satisfying the distributional constraint, so

$$F(\mathbb{T}(R')) = \frac{\sum_{c_i \in B} a_i}{nM}, \quad \text{hence} \quad F(\mathbb{T}(R')) \geq \tau^* \iff \sum_{c_i \in B} a_i \geq W.$$

Since all solutions contain exactly k tables, minimizing average unionability is equivalent to minimizing the corresponding sum (Table 1). The repair cost is therefore

$$L(B) = \sum_{t_i \in A} U(Q, t_i) - \sum_{c_i \in B} U(Q, c_i) = 0 - \sum_{c_i \in B} (-a_i) = \sum_{c_i \in B} a_i.$$

Thus the repair problem seeks a feasible subset $B \subseteq C$ of minimum total value:

$$\min \sum_{c_i \in B} a_i \quad \text{s.t.} \quad \sum_{c_i \in B} a_i \geq W.$$

Every feasible repair has cost at least W . The optimum equals W if and only if some subset of $\{a_1, \dots, a_n\}$ sums to exactly W . Therefore, the original SUBSET SUM instance is a Yes-instance if and only if the constructed repair instance has optimal cost exactly W .

Since the construction is polynomial, computing a minimum-cost repair is NP-hard. $\square$

8.1 Greedy Swap

Since exact search over all injective matchings is intractable, we reduce the search space with a greedy strategy. Rather than enumerating all possible subsets of swaps, we process violating tables sequentially: at each step, we select the single swap—replacing one violating table with one candidate—that maximally improves distributional balance at minimum unionability cost, and commit to it before moving to the next. This raises a natural question: can we rank the candidate pool C once and reuse that ranking across all violating tables, reducing each step to a simple lookup? Claim 2 rules this out, showing that the optimal candidate is specific to the violating table at hand and cannot be determined independently of it. This drives the design of our algorithm: candidates must be evaluated separately for each violating table rather than resolved by a global ranking.

CLAIM 2 (TABLE-DEPENDENT OPTIMAL REPLACEMENT). *Let*

$$c^*(t) = \arg \max_{c \in C} F(\mathbb{T}(R(t, c)), V_D, \mathcal{M}).$$

There may exist two violating tables $t, t' \in R_{vi}$ such that $c^(t) \neq c^*(t')$.*

PROOF. For $t \in R_{vi}$, let $N_0^{(t)}$ be the total number of tuples in $\mathcal{M}$ contributed by all tables in $R \setminus \{t\}$, let $W_0^{(t)}$ be the total number of tuples *not* in $\mathcal{M}$ contributed by all tables in $R \setminus \{t\}$ (i.e., $W_0^{(t)} = \sum_{T_i \in R \setminus \{t\}} (n_i - N_i)$), and let $B_t = (N_0^{(t)}, W_0^{(t)})$. Let $c_1, c_2 \in C$ be candidate tables with $N(c_1) \neq N(c_2)$ or $n(c_1) - N(c_1) \neq n(c_2) - N(c_2)$.

Let

$$D(B_t) = F(\mathbb{T}(R(t, c_1))) - F(\mathbb{T}(R(t, c_2))).$$

Substituting and simplifying yields

$$D(B_t) = \frac{\mathbb{A} \times N_0^{(t)} + \mathbb{B} \times W_0^{(t)} + \mathbb{C}}{(N_0^{(t)} + W_0^{(t)} + n(c_1)) \times (N_0^{(t)} + W_0^{(t)} + n(c_2))}$$

for constants $\mathbb{A} = (n(c_2) - N(c_2)) - (n(c_1) - N(c_1))$, $\mathbb{B} = N(c_1) - N(c_2)$, and $\mathbb{C} = N(c_1)(n(c_2) - N(c_2)) - N(c_2)(n(c_1) - N(c_1))$. At least one of $\mathbb{A}$ or $\mathbb{B}$ is nonzero. The denominator is positive, and the numerator is a linear function of $N_0^{(t)}$ and $W_0^{(t)}$ plus a constant. Different violating tables induce different $N_0^{(t)}$ and $W_0^{(t)}$; there exists a threshold line in the $(N_0^{(t)}, W_0^{(t)})$ -plane where $D(B_t) = 0$. Choosing two violating tables t_1, t_2 whose induced $(N_0^{(t_1)}, W_0^{(t_1)})$ and $(N_0^{(t_2)}, W_0^{(t_2)})$ lie on opposite sides of this line yields $D(B_{t_1}) > 0$ and $D(B_{t_2}) < 0$, hence

$$F(\mathbb{T}(R(t_1, c_1))) > F(\mathbb{T}(R(t_1, c_2))) \quad \text{and} \quad F(\mathbb{T}(R(t_2, c_1))) < F(\mathbb{T}(R(t_2, c_2)))$$

Therefore the preferred replacement candidate depends on the violating table. $\square$

We employ heuristic strategies to efficiently identify high-quality sequences of replacements. Since each swap might reduce the unionability score, we aim to minimize the number of swaps. In other words, for each swap we perform, we seek to achieve the largest possible improvement in distributional balance while incurring the smallest possible loss in unionability.

Algorithm 1: Greedy_swap

Input: R : top- k result returned by search system; C : candidate tables; $\mathcal{F}^*$; δ

Output: R : distributionally balanced unionable tables

```

1  $\Delta(\mathcal{F}^*, F(\mathbb{T}(Q \cup R), V_D, \mathcal{M})) > \delta$   $R_{vi} \leftarrow \text{getViolating}(R)$ ;
2  $R_{vi}.\text{sort}()$ ;
3 foreach  $t \in R_{vi}$  do
4    $b \leftarrow \text{findBest}(C, t)$  // swap balances the global
     distribution
5   if  $F(\mathbb{T}(Q \cup R)) < F(\mathbb{T}(Q \cup R(t, b)))$  then
6      $R \leftarrow R(t, b)$ ;
7      $C \leftarrow C \setminus \{b\}$ ;
8   if  $\Delta(\mathcal{F}^*, F(\mathbb{T}(R))) \leq \delta$  then
9     return  $R$ 
10 return  $\emptyset$  // result could not be achieved
```

Three modules of Algorithm 1 require further clarification:

- The strategy used to sort the violating tables R_{vi} (Line 2) is based on two ordering criteria applied in descending order. First, tables are ranked according to their distributional gain, defined as the improvement in F achieved by removing the table from the result set, i.e., $F(\mathbb{T}(R \setminus \{t\})) - F(\mathbb{T}(R))$. In the case of ties, tables are further ordered by the negative of their unionability score $U(Q, t)$, thereby prioritizing tables whose removal results in lower unionability loss.
- The procedure used to select the best replacement candidate from the candidate pool C (Line 4) considers all candidate tables for each violating table $t \in R_{vi}$. For every candidate table, we evaluate the resulting proportion after performing the swap. The candidate that yields the maximum improvement in F is selected. In the case of ties, preference is given to the candidate that minimizes unionability loss.
- A swap is performed only if the resulting proportion of the updated result set strictly improves after the replacement (Line 7).

Algorithm 1 is sound and efficient. The algorithm returns a non-empty result only by certifying $\Delta(\mathcal{F}^*, F(\mathbb{T}(R'))) \leq \delta$, and every committed swap strictly increases F by construction, since a replacement is applied only when it yields a strict improvement. For time complexity, the outer loop iterates over at most $|R_{vi}|$ violating tables, and findBest evaluates each candidate in C in $O(1)$ time given precomputed N_i and n_i values, giving an overall runtime of $O(|R_{vi}| \cdot |C|)$.

There are two additional discussions that would strengthen this section, though I am leaving them for future revisions. First, it would be useful to

include a concrete example illustrating that the algorithm can fail to find a solution even when a valid solution exists, thereby clarifying that it is not complete. Second, it would be helpful to characterize the conditions on the input under which the algorithm is guaranteed to find a solution, providing readers with a clearer understanding of its applicability and limitations. –bk

8.2 Preference-Based Filtering

Although $O(|R_{vi}| \cdot |C|)$ is already a significant reduction over exhaustive search, it remains costly when either $|R_{vi}|$ or $|C|$ is large—a realistic scenario in data lakes with thousands of candidate tables. We therefore introduce a filtering step that prunes C prior to the main loop, reducing the number of candidates each call to `findBest` must evaluate without sacrificing solution quality. As discussed in Claim 2, the ordering of tables in the candidate pool generally depends on the particular violating table being replaced. However, some conditions hold independently of the specific violating table and can therefore be used to prune candidates from C in advance. To formalize these conditions, we use the notion of preferences as introduced in [5].

8.2.1 Preference Formulas. Chomicki [5] proposes a logical framework for expressing user preferences in relational queries. A *preference relation* $\succ$ over a set of tuples is a binary relation specified by a first-order *preference formula* $C(t_1, t_2)$, where $t_1 \succ t_2$ denotes that t_1 is strictly preferred to t_2 . A tuple t is *dominated* if there exists another tuple t' with $t' \succ t$; otherwise t is *non-dominated*. The central operator of the framework is the *winnow* operator $\omega_C(r)$, which returns the set of non-dominated tuples from a relation r :

$$\omega_C(r) = \{t \in r \mid \nexists t' \in r : t' \succ t\}.$$

When $\succ$ is a strict partial order (irreflexive and transitive), $\omega_C(r)$ is guaranteed to be non-empty for any finite, non-empty r [5]. The key property that makes winnow useful for pruning is its safety guarantee: any dominated tuple can be discarded without affecting the optimal answer, regardless of the query context. Chomicki proposes several algorithms for computing $\omega_C(r)$, including a nested-loop algorithm that is correct for any preference relation without requiring additional structural assumptions. We exploit this framework directly: by instantiating $\succ$ as the dominance relation over candidate tables (Equation 3), we use winnow to prune C to a smaller pool C' of non-dominated candidates, with the guarantee that no pruned candidate can be part of an optimal repair. Given two candidate tables c_1 and c_2 , and an arbitrary violating table $t \in R_{vi}$ to be replaced, we define the preference relation over candidate tables as follows:

$$\begin{aligned} c_1 \succ c_2 \equiv & (F(\mathbb{T}(R(t, c_1))) > F(\mathbb{T}(R(t, c_2)))) \wedge U(Q, c_1) \geq U(Q, c_2) \\ & \vee (F(\mathbb{T}(R(t, c_1))) = F(\mathbb{T}(R(t, c_2)))) \wedge U(Q, c_1) > U(Q, c_2). \end{aligned} \quad (3)$$

We next establish conditions under which $F(\mathbb{T}(R(t, c_1))) > F(\mathbb{T}(R(t, c_2)))$ holds regardless of the particular table being replaced, that is, regardless of the choice of t .

CLAIM 3. *Given two candidate tables c_1 and c_2 , and an arbitrary violating table $t \in R_{vi}$, the following conditions hold:*

$$\begin{aligned} N(c_1) > N(c_2) \wedge n(c_1) - N(c_1) < n(c_2) - N(c_2) &\Rightarrow F(\mathbb{T}(R(t, c_1))) > F(\mathbb{T}(R(t, c_2))) \\ N(c_1) > N(c_2) \wedge n(c_1) - N(c_1) = n(c_2) - N(c_2) &\Rightarrow F(\mathbb{T}(R(t, c_1))) > F(\mathbb{T}(R(t, c_2))) \\ N(c_1) = N(c_2) \wedge n(c_2) - N(c_2) > n(c_1) - N(c_1) &\Rightarrow F(\mathbb{T}(R(t, c_1))) > F(\mathbb{T}(R(t, c_2))) \\ N(c_1) = N(c_2) \wedge n(c_1) - N(c_1) = n(c_2) - N(c_2) &\Rightarrow F(\mathbb{T}(R(t, c_1))) = F(\mathbb{T}(R(t, c_2))) \end{aligned}$$

PROOF. Adopting the notation of Claim 2, we consider each condition in turn. The first, second, and third conditions each imply $D(B_t) > 0$ regardless of the choice of t . The fourth condition implies $D(B_t) = 0$. $\square$

Applying Claim 3, we can prune dominated candidates from C before invoking the main swap loop. Without imposing any additional structure on the preference formula or the preference relation $\succ$, we adopt the classical nested-loop dominance filter of [5] (Algorithm 2), which removes any candidate c_i for which there exists a c_j with $c_j \succ c_i$. Although the filter requires $O(|C|^2)$ comparisons in the worst case, this cost is incurred only once as a preprocessing step, while the pruned pool C' is reused across all $|R_{vi}|$ iterations of Algorithm 1. When $|C|$ is large, the reduction in candidate pool size can therefore yield a substantial net saving.

Algorithm 2: Candidate Filtering Nested Loop

```

1 NestedLoop( $C$ )  $C' \leftarrow C$ ;
2 foreach  $c_i \in C$  do
3   foreach  $c_j \in C$  do
4     if  $c_j \succ c_i$  then
5        $C' \leftarrow C' \setminus \{c_i\}$ ;
6     break;
7 return  $C'$ ;

```

There are multiple filtering algorithms that are more efficient than a simple nested loop (Algorithm 2). However, for these algorithms to work, the preference properties must satisfy certain conditions. The first approach is the blocked nested-loop algorithm [1], which is applicable when the preference relation is a strict partial order. We next show that our preference relation satisfies this property.

CLAIM 4. *The preference relation defined in Equation (3) is a strict partial order.*

PROOF. We need to show that the relation is irreflexive and transitive.

Irreflexivity. We show that for every candidate c , $c \not\succ c$. Assume, for contradiction, that there exists a candidate c such that $c \succ c$. By the definition of $\succ$ in Equation (3), this would require either $F(\mathbb{T}(R(t, c))) > F(\mathbb{T}(R(t, c)))$, or $F(\mathbb{T}(R(t, c))) = F(\mathbb{T}(R(t, c)))$ and $U(Q, c) > U(Q, c)$. Both are impossible. Therefore $c \not\succ c$ for all c , and hence the relation is irreflexive.

Transitivity. We show that $\forall c_1, c_2, c_3 : c_1 \succ c_2 \wedge c_2 \succ c_3 \Rightarrow c_1 \succ c_3$. There are multiple cases; we present one, and the others follow similarly.

• **Case I.**

$$c_1 \succ c_2 \Rightarrow F(\mathbb{T}(R(t, c_1))) > F(\mathbb{T}(R(t, c_2)))$$

$$c_2 \succ c_3 \Rightarrow F(\mathbb{T}(R(t, c_2))) > F(\mathbb{T}(R(t, c_3)))$$

then by transitivity of $>$,

$$F(\mathbb{T}(R(t, c_1))) > F(\mathbb{T}(R(t, c_3))).$$

Also, from $U(Q, c_1) \geq U(Q, c_2)$ and $U(Q, c_2) \geq U(Q, c_3)$, we obtain $U(Q, c_1) \geq U(Q, c_3)$. Hence,

$$F(\mathbb{T}(R(t, c_1))) > F(\mathbb{T}(R(t, c_3))) \wedge U(Q, c_1) \geq U(Q, c_3),$$

which implies $c_1 \succ c_3$.

The other cases can be shown similarly. $\square$

Algorithm 3: Blocked Nested Loop

```

1 BlockedNestedLoop( $C$ )  $I \leftarrow C$ ;
2 while  $I \neq \emptyset$  do
3    $W \leftarrow \emptyset$ ;
4    $F_{\text{out}} \leftarrow \emptyset$ ;
5   foreach  $t \in I$  do
6     if  $\exists w \in W : w \succ t$  then
7       continue;
8     else if  $\exists w \in W : t \succ w$  then
9        $W \leftarrow W \setminus \{w \in W \mid t \succ w\}$ ;
10       $W \leftarrow W \cup \{t\}$ ;
11     else
12       if  $|W| < B$  then
13          $W \leftarrow W \cup \{t\}$ ;
14       else
15          $F_{\text{out}} \leftarrow F_{\text{out}} \cup \{t\}$ ;
16   if  $F_{\text{out}} = \emptyset$  then
17     foreach  $w \in W$  do
18       output  $w$ ;
19    $I \leftarrow F_{\text{out}}$ ;
```

Even when all data fits in memory, blocked nested loop can outperform a simple nested loop by maintaining only the current non-dominated candidates and discarding dominated tuples early. However, its benefit depends on the size of the incomparable set; if many tuples are mutually incomparable, the gain may be limited. We study this further in the experiments.

ACKNOWLEDGMENTS

This work was supported by the [...] Research Fund of [...] (Number [...]). Additional funding was provided by [...] and [...]. We also thank [...] for contributing [...].

REFERENCES

[1] Stephan Börzsönyi, Donald Kossmann, and Konrad Stocker. 2001. The Skyline Operator. In *Proceedings of the 17th International Conference on Data Engineering, April 2-6, 2001, Heidelberg, Germany*, Dimitrios Georgakopoulos and Alexander Buchmann (Eds.). IEEE Computer Society, 421–430. <https://doi.org/10.1109/ICDE.2001.914855>

[2] Alberto Caprara, Hans Kellerer, Ulrich Pferschy, and David Pisinger. 2000. Approximation algorithms for knapsack problems with cardinality constraints. *Eur. J. Oper. Res.* 123, 2 (2000), 333–345. [https://doi.org/10.1016/S0377-2217\(99\)00261-1](https://doi.org/10.1016/S0377-2217(99)00261-1)

[3] L. Elisa Celis, Damian Straszak, and Nisheeth K. Vishnoi. 2018. Ranking with Fairness Constraints. In *45th International Colloquium on Automata, Languages, and Programming, ICALP 2018, Prague, Czech Republic, July 9-13, 2018 (LIPIcs)*, Ioannis Chatzigiannakis, Christos Kaklamani, Dániel Marx, and Donald Sannella (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 28:1–28:15. <https://doi.org/10.4230/LIPICS.ICALP.2018.28>

[4] Abraham Charnes and William W Cooper. 1962. Programming with linear fractional functionals. *Naval Research logistics quarterly* 9, 3-4 (1962), 181–186.

[5] Jan Chomicki. 2003. Preference formulas in relational queries. *ACM Trans. Database Syst.* 28, 4 (2003), 427–466. <https://doi.org/10.1145/958942.958946>

[6] Werner Dinkelbach. 1967. On nonlinear fractional programming. *Management science* 13, 7 (1967), 492–498.

[7] Cynthia Dwork, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Richard S. Zemel. 2012. Fairness through awareness. In *Innovations in Theoretical Computer Science 2012, Cambridge, MA, USA, January 8-10, 2012*, Shafi Goldwasser (Ed.). ACM, 214–226. <https://doi.org/10.1145/2090236.2090255>

[8] Grace Fan, Jin Wang, Yuliang Li, Dan Zhang, and Renée J. Miller. 2023. Semantics-aware Dataset Discovery from Data Lakes with Contextualized Column-based Representation Learning. *Proc. VLDB Endow.* 16, 7 (2023), 1726–1739. <https://doi.org/10.14778/3587136.3587146>

[9] Besat Kassaie and Renée J. Miller. 2026. Novel Table Search [Technical Report]. *CoRR* abs/2603.07235 (2026). <https://doi.org/10.48550/ARXIV.2603.07235>

[10] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836.

[11] Tomomi Matsui, Yasufumi Saruwatari, and Maiko Shigeno. 1992. An analysis of dinkelbach’s algorithm for 0-1 fractional programming problems. *METR92-14* (1992).

[12] Fengqi You, Pedro M. Castro, and Ignacio E. Grossmann. 2009. Dinkelbach’s algorithm as an efficient method to solve a class of MINLP models for large-scale cyclic scheduling problems. *Comput. Chem. Eng.* 33, 11 (2009), 1879–1889. <https://doi.org/10.1016/J.COMPHEMENG.2009.05.014>

[13] Erkang Zhu, Dong Deng, Fatemeh Nargesian, and Renée J. Miller. 2019. JOSIE: Overlap Set Similarity Search for Finding Joinable Tables in Data Lakes. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*, Peter A. Boncz, Stefan Manegold, Anastasia Ailamaki, Amol Deshpande, and Tim Kraska (Eds.). ACM, 847–864. <https://doi.org/10.1145/3299869.3300065>